

AutoGUI-v2: A Comprehensive Multi-Modal GUI Functionality Understanding Benchmark

Hongxin Li^{*1,2,3} Xiping Wang^{*5} Jingran Su⁵ Zheng Ju^{1,2,3} Yuntao Chen⁴ Qing Li⁵
Zhaoxiang Zhang^{1,2,3,6}✉

¹University of Chinese Academy of Sciences (UCAS)

²New Laboratory of Pattern Recognition (NLPR), CASIA

³State Key Laboratory of Multimodal Artificial Intelligence Systems (MAIS), CASIA

⁴Hong Kong Institute of Science & Innovation, CASIA

⁵PolyU ⁶Shanghai Artificial Intelligence Laboratory

Code: <https://github.com/ZJULiHongxin/UIPro>

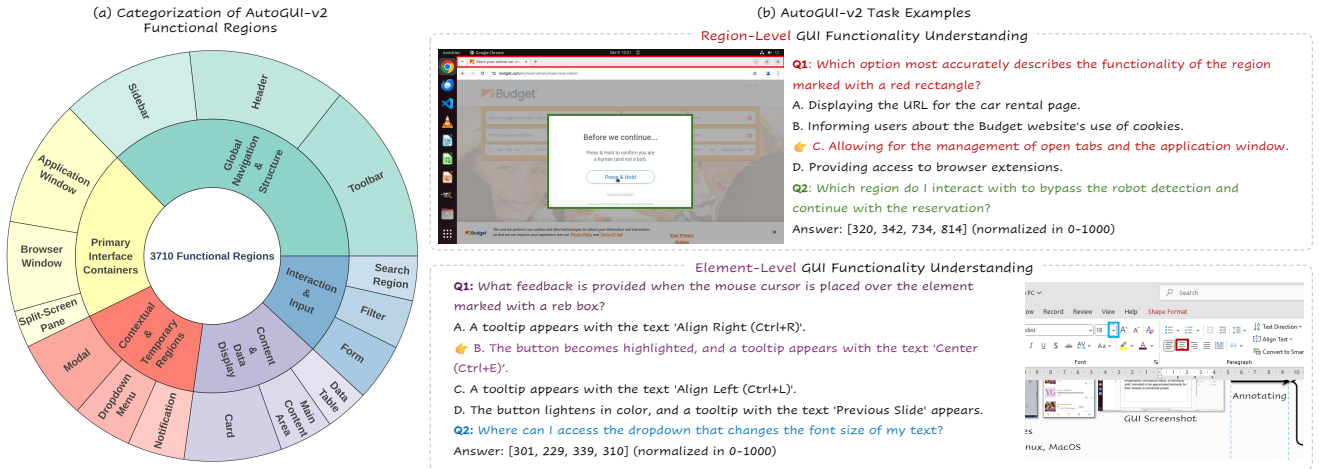


Figure 1. **AutoGUI-v2 benchmark overview.** (a) Region types covered. (b) Representative functionality understanding tasks.

Abstract

Autonomous agents capable of navigating Graphical User Interfaces (GUIs) hold the potential to revolutionize digital productivity. However, achieving true digital autonomy extends beyond reactive element matching; it necessitates a predictive mental model of interface dynamics and the ability to foresee the “digital world state” resulting from interactions. Despite the perceptual capabilities of modern Vision-Language Models (VLMs), existing benchmarks remain bifurcated—focusing either on black-box task completion or static, shallow grounding—thereby failing to assess whether agents truly comprehend the implicit functionality and transition logic of GUIs. To bridge this gap, we introduce **AutoGUI-v2**, a comprehensive benchmark designed to evaluate deep GUI functionality understanding and in-

teraction outcome prediction. We construct the benchmark using a novel VLM-human collaborative pipeline that recursively parses multi-platform screenshots into hierarchical functional regions to generate diverse evaluation tasks. Providing 2,753 tasks across six operating systems, AutoGUI-v2 rigorously tests agents on region and element-level semantics, grounding, and dynamic state prediction. Our evaluation reveals a striking dichotomy in VLMs: while open-source models fine-tuned on agent data (e.g., Qwen3-VL) excel at functional grounding, commercial models (e.g., Gemini-2.5-Pro-Thinking) dominate in functionality captioning. Crucially, all models struggle with complex interaction logic of uncommon actions, highlighting that deep functional understanding remains a significant hurdle. By systematically measuring these foundational capabilities, AutoGUI-v2 offers a new lens for advancing the next generation of GUI

* Co-authors. ✉ Corresponding author.

agents.

1. Introduction

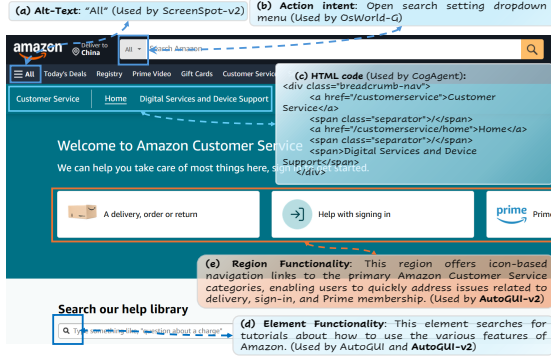


Figure 2. AutoGUI-v2 provides rich functional semantics for both GUI elements and regions compared with existing benchmarks.

The development of autonomous agents capable of navigating digital interfaces represents a transformative frontier in artificial intelligence, promising to redefine human-computer interaction [6, 10, 12–14, 16, 20, 31, 51, 52, 57, 59]. While recent Vision-Language Models (VLMs) [5, 21, 40, 44, 47] have empowered agents to interact with GUIs across diverse platforms, effective digital autonomy requires more than reactive element grounding [15, 35, 61]. It demands a profound understanding of interface dynamics, the implicit functionality of regions, and the ability to predict the “digital world state” resulting from an action [15]. Such predictive reasoning is a hallmark of human competence and is essential for agents to generalize across diverse scenarios [1, 34, 50].

Despite this requirement, existing benchmarks largely overlook this deeper functional understanding, falling into two primary categories. The first, *task-completion benchmarks* [8, 26, 32, 33, 53, 63], treat the agent as a black box, assessing success rates without diagnosing how the agent interprets GUI functionality. The second, *grounding benchmarks* [6, 15, 17, 22, 30, 54], offer fine-grained evaluation but are limited to simplistic element localization. These benchmarks typically rely on brief appearance descriptions [6, 19], alt-texts [22], or action intents [54] and fail to test the understanding of transition logic or GUI context, as illustrated in Fig. 2. While the recent AutoGUI [15] introduced functionality understanding evaluation, it remains constrained by platform diversity and lacks tasks probing transition dynamics. Consequently, a critical gap remains: the field lacks a benchmark that measures an agent’s foundational model of GUI functionality and interaction outcomes.

To bridge this gap, we introduce **AutoGUI-v2**, a comprehensive benchmark designed to evaluate a VLM-based

GUI agent’s deep understanding of both element-level and region-level functionality. AutoGUI-v2 is constructed using a novel VLM-human collaborative pipeline. We utilize **Gemini-2.5-Pro-Thinking** [39] to recursively divide multi-platform screenshots into hierarchical functional regions, which are subsequently verified by a VLM-based scorer and refined via manual annotation to ensure high precision. Leveraging these hierarchical divisions, we generate challenging tasks that require agents to localize targets based on functional descriptions and predict interaction outcomes. Complementing this, we employ **OmniParser-v2** [27] to assist in element-level grounding and captioning task generation. In total, AutoGUI-v2 comprises **2,753** evaluation tasks across six operating systems, serving as a rigorous testbed for GUI functionality comprehension.

Evaluation on AutoGUI-v2 reveals a striking divergence in capabilities: Open-source models (e.g., GLM-4.5, Qwen3-VL) unexpectedly outperform leading commercial models (e.g., Gemini-2.5-Pro-Thinking) at functionality-oriented grounding—the task of localizing “where” a function is. Conversely, these same commercial models dominate the functionality captioning task, or reasoning “what” a region/element does. Furthermore, our analysis pinpoints why models fail. Performance plummets for irregular region types and complex interactions, indicating the models rely on overt cues and fail to grasp implicit functionality. Finally, these models are consistently tricked by “hard” plausible distractors (i.e., visually similar but functionally distinct candidates) in our functionality captioning tasks, proving their failures stem from a lack of context-aware functionality understanding, not random error.

Our contributions are summarized as follows:

- We introduce a scalable pipeline for detecting and annotating hierarchical functional regions across multi-platform GUIs, providing a valuable resource of region-level annotations to the community.
- We propose **AutoGUI-v2**, a large-scale benchmark evaluating deep GUI functionality understanding at both region and element levels, moving beyond simple grounding to assessing the comprehension of GUI dynamics.
- We provide a comprehensive analysis of leading VLMs, offering unprecedented insights into their limitations in understanding GUI functionality and state transitions.

2. Related Works

2.1. GUI Benchmarks

The evolution of GUI benchmarking has progressed through three distinct paradigms. The foundation lies in *grounding capability evaluation*, which tests an agent’s ability to localize UI elements from natural language instructions; this was pioneered by ScreenSpot [6] and expanded by high-resolution and multi-application benchmarks such as

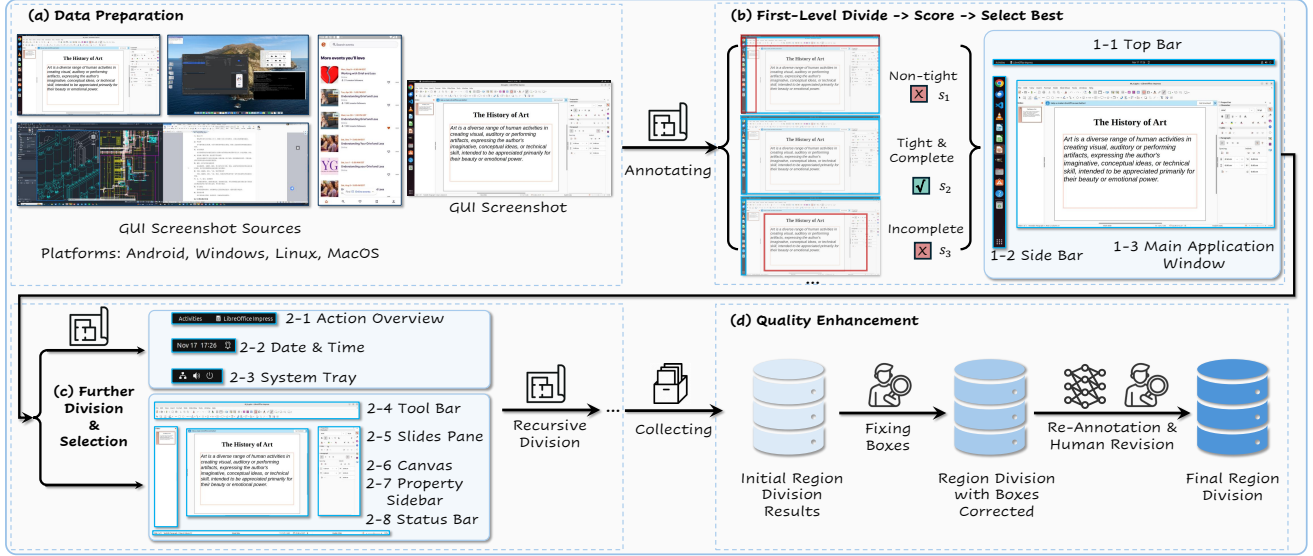


Figure 3. Overview the AutoGUI-v2 Annotation Pipeline. (a) The process begins by sourcing diverse, multi-platform GUI screenshots. (b) A VLM (Gemini-2.5-Pro-Thinking) proposes an initial, first-level decomposition into functional regions, which are automatically scored for quality. (c) This “divide-and-verify” process is applied recursively, progressively breaking down complex regions into their granular, non-dividable components. (d) In the final stage, human experts provide pixel-perfect refinement of all bounding boxes, enabling a final VLM pass to generate the high-fidelity functionality descriptions.

Table 1. Comparing our AutoGUI-v2 dataset with existing popular GUI understanding benchmarks.

Benchmark	#Operating Systems	Functionality Annotation Type	Max Res.	Target Type	#Tasks
ScreenSpot [6]	5	N/A	1280 × 720	Element	1272
ScreenSpot-Pro [17]	3	N/A	3840 × 2160	Element	1581
VisualWebBench [22]	1	N/A	3840 × 2160	Element	1581
OSWorld-G [55]	1	N/A	1920 × 1080	Element	510
UI-Vision [30]	1	N/A	1920 × 1080	Element & Region	8227
MMBench-GUI [48]	6	N/A	3840 × 2160	Element	8000+
GUI-Knowledge-Bench [35]	6	Brief	3840 × 2160	Element	3483
AutoGUI [15]	2	Contextual	1280 × 720	Element	2000
AutoGUI-v2 (ours)	6	Contextual	3840 × 2160	Element & Region	2753

ScreenSpot Pro [17], UI-Vision [30], and OSWorld-G [54]. Subsequently, *offline agent evaluation* emerged to assess agents on static environment snapshots, exemplified by Mind2Web [8] for web, GUI Odyssey [26] for mobile, and the cross-platform GUICourse [4]. Most recently, the field has advanced to *online agent evaluation* in dynamic environments via tools like OSWorld [53], AndroidWorld [33], AndroidLab [56], MacOSWorld [58], and WAA [2]. However, these benchmarks typically overlook the understanding of interaction-induced state changes, failing to measure if agents grasp essential GUI dynamics. AutoGUI-v2 fills this gap by benchmarking comprehensive GUI functionality understanding. Unlike the concurrent GUI Knowledge Bench [35], which addresses interaction outcomes but lacks region-level granularity and sufficient context for element localization, our approach ensures a robust evaluation of underlying GUI functionality.

2.2. Multi-Modal GUI Agents

The development of multimodal GUI agents has shifted from API-dependent systems like AppAgent [62], Sheet-Copilot [14], and OmniParser [27] to scalable, end-to-end approaches—such as CogAgent [12], ShowUI [20], UGround [10], SpiritSight [13], Aguis [57], Jedi [55], OS-Atlas [52], GUI-Xplore [37], and UI-Tars [31]. Recently, these visual methods have been augmented by reinforcement learning paradigms [23, 28, 38], which enhance generalization and reduce data reliance in complex environments. However, while agent architectures have advanced, evaluation methodologies remain fixated on task execution rates, neglecting the foundational understanding of GUI dynamics. Our AutoGUI-v2 addresses this gap by probing the underlying “digital state prediction” useful for agent ability [34].

3. AutoGUI-v2 Construction

This section details the AutoGUI-v2 annotation pipeline (Fig. 3) and task generation methodology (Fig. 4).

3.1. Definitions

A **functional region** is defined as a high-level grouping of individual UI elements (such as buttons, input fields, links, navigation elements, or icons) that collectively serve a unified purpose. These regions are organized hierarchically, where parent regions are recursively decomposed into granular child units (Fig. 3). The region types are exemplified in

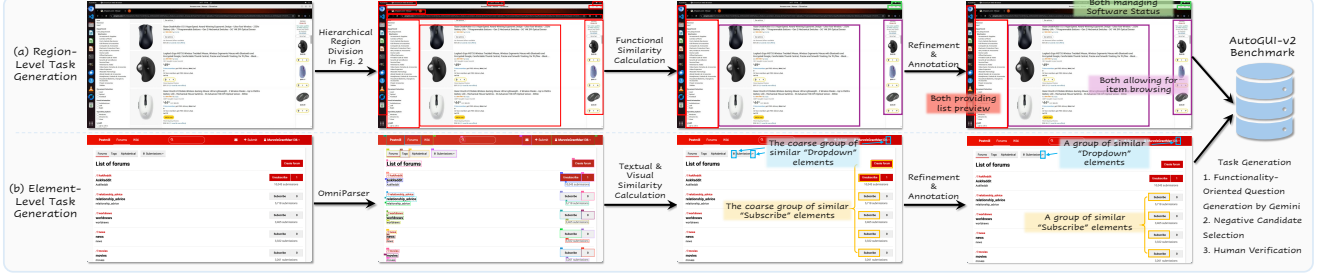


Figure 4. The pipeline of generating AutoGUI-v2 benchmark tasks introduced in Sec. 3.3.2 and Sec. 3.4.

Fig. 1 and detailed in Sec. 6.5 of Appendix.

At the element level, we focus on **visually similar elements with distinct functionality**. These visually analogous yet functionally distinct components force models to rely on context rather than appearance alone.

3.2. Data Sources

We aggregate GUI screenshots from diverse sources, including ScreenSpot-Pro [17], OSWorld-G [54], MMBench-GUI [48], AgentNet [49], and AMEX [3]. This multi-source strategy ensures robust coverage across desktop, web, and mobile domains (Details in Appendix).

3.3. Region-Level Functionality Understanding

To construct high-quality, hierarchical annotations, we employ a three-stage human-in-the-loop pipeline.

3.3.1. Hierarchical Region Annotation Pipeline

Stage 1: Automated Proposal & Verification. Drawing inspiration from the AutoGUI pipeline [15], we leverage *Gemini-2.5-Pro-Thinking* [39] to automatically annotate functional regions through a multi-step process. Given a region image $\mathbf{I}_{region} \in \mathbb{R}^{H \times W \times 3}$, the VLM identifies K first-level sub-regions, predicting each region’s functionality $\{F_i\}$ with instance-specific contextual details, layout description $\{D_i\}$, normalized bounding boxes $\{B_i\}$, and a divisibility flag $\{d_i\}$. This process is formulated as:

$$\{F_i, D_i, B_i, d_i\}_{i=1}^K = \text{VLM}(\mathbf{I}_{region}, p_{anno}) \quad (1)$$

where p_{anno} is the annotation prompt shown in Appendix. Afterwards, this pipeline recursively divides the regions with $d_i = \text{True}$ until all regions are no longer dividable.

To mitigate bounding box errors (examples in Appendix), we incorporate a **refinement mechanism**. A separate VLM instance evaluates the cropped sub-region $\mathbf{I}_i$ against the root screenshot $\mathbf{I}_{root}$ on two metrics: (1) **Completeness** $s_i^{comp} \in \{0, 1\}$, checking for full visibility, and **Boundedness** $s_i^{bound} \in \{0, 1\}$, ensuring tight framing.

$$s_i^{comp}, s_i^{bound} = \text{VLM}(\mathbf{I}_{root}, \mathbf{I}_i, p_{scoring}) \quad (2)$$

where $p_{scoring}$ is the scoring prompt shown in Appendix.

For an empirically better tradeoff of cost and quality, if a proposal obtains the average scores $\sum_{i=1}^K s_i^{comp} / K \geq 0.8$ and $\sum_{i=1}^K s_i^{bound} / K \geq 0.8$, the proposal is accepted; otherwise, the model regenerates up to $N = 3$ iterations. Note that this mechanism is not intended for perfect division, but to provide accurate initial divisions for human correction in the next stage (More details in the appendix).

Stage 2: Human-in-the-Loop Bounding Correction. To achieve benchmark-grade precision, the authors refine the automated proposals using a custom web interface (details in Sec. 6.4). This tool enables annotators to traverse region trees and manually adjust bounding boxes B_i to tightly encompass the regions, correcting strictly for spatial accuracy.

Stage 3: Functionality Re-annotation. Since manual spatial corrections alter the visual content of the crops, we perform a final automated pass to align the semantic annotations with the refined regions. The corrected crop $\mathbf{I}_i^{fixed}$ is fed back into the VLM to generate updated functionality F_i^{fixed} and descriptions D_i^{fixed} :

$$F_i^{fixed}, D_i^{fixed} = \text{VLM}(\mathbf{I}_{root}, \mathbf{I}_i^{fixed}, p_{reanno}) \quad (3)$$

Finally, human annotators validate these re-annotations against our quality criteria (i.e., Contextuality, High-Level Purpose, and Non-Hallucination) explained in Sec. 6.4.

3.3.2. Task Generation

We generate evaluation tasks by identifying groups of visually similar regions that serve distinct functions.

Stage 1: Semantic Clustering. We initialize candidate groups by computing a cosine similarity matrix on the embeddings of region descriptions (D_i), generated via Qwen3-Embedding [64]. Regions with similarity scores $s_{sem} > th_{sem}$ are clustered, effectively grouping regions that share visual attributes (e.g., layout) as described in text space.

Stage 2: Group Verification and Refinement. A VLM (i.e. Gemini-2.5-Pro-Thinking) verifies these initial clusters within the global screenshot context to: 1) remove visually dissimilar false positives; 2) confirm that grouped regions possess distinct functionalities; and 3) downsample over-

Table 2. Statistics of AutoGUI-v2 evaluation tasks.

Task Type	#Tasks	#Avg. Words of Questions	Top-3 Res.
Region-Level Grounding	442	20.2	1920x1080: 50.9%, 1280x720: 22.2%, 3840x2160: 5.9%
Region-Level Captioning	447	60.0	1920x1080: 50.1%, 1280x720: 21.7%, 3840x2160: 5.8%
Element-Level Grounding	1076	24.3	1920x1080: 28.3%, 3840x2160: 22.8%, 2560x1440: 22.8%
Element-Level Captioning	788	96.0	1920x1080: 33.4%, 2560x1440: 21.7%, 3840x2160: 20.3%

sized clusters ($N > 5$) to retain only the most representative and confusing instances.

Stage 3: Task Construction. Using these refined similarity groups, we construct two challenge types: a) **Functionality-Oriented Grounding.** We generate questions requiring the agent to localize a target region by outputting its bounding box based on an functionality requirement. We randomly select a target region from each similarity group and generate a question. For example, “Which region should you interact to switch workspaces?”, which strictly avoids appearance-based cues. b) **Functionality-Oriented Captioning.** We formulate multi-choice questions predicting interaction outcome. For example, “What happens if I interact with the highlighted vertical bar?” Options include the ground truth, *hard negatives* (outcomes of visually similar regions in the same similarity group), and *easy negatives* (random screen functions). Answer positions are randomized to prevent bias.

3.4. Element-Level Functionality Understanding

The element-level pipeline (Fig. 4) mirrors the region-level methodology but adapts the element discovery phase to handle fine-grained GUI components.

Detection and Grouping. Unlike the recursive VLM division used for regions, we employ **OmniParser-v2** [27] to detect all interactive elements. To identify visually similar candidates, we compute a cosine similarity matrix on visual embeddings extracted via **DINO-v3** [36]. To remove false positives, we retain pairs with high textual similarity ($s_{text} > th_{text}$) using fuzzy matching¹ and consolidate clusters via a Disjoint Set Union (DSU) structure [9].

Refinement and Task Construction. The candidate groups undergo the same VLM-based verification and annotation process described in Sec. 3.3.2 to ensure functional distinctness. Finally, we generate grounding and captioning tasks using the identical formulation strategies defined in Sec. 3.3.2. These questions are tailored to element-specific interactions (e.g., “double-click”, “long-press”), challenging the model to distinguish between the target and “hard negative” elements that look similar but behave differently.

3.5. Dataset Statistics

Our pipeline yields 3,710 hierarchical functional regions, enabling 2,509 evaluation tasks (889 region-level, 1,620 element-level). As shown in Tab. 1 and 2, the benchmark spans multiple platforms and resolutions, testing diverse, rich functional semantics (Fig. 1). Further statistical details

¹<https://github.com/rapidfuzz/RapidFuzz>

and cost analysis are provided in Appendix Sec. 6.6. Task samples can be viewed in the *supplementary materials*.

4. Experiments

With AutoGUIv2, we rigorously evaluate whether existing VLMs can accurately understand the contextual functionality of GUI regions and elements.

4.1. Experimental Settings

Evaluated Models. We evaluate a comprehensive suite of commercial and open-source VLMs, categorized as follows: **a) Commercial Models:** Leading VLMs accessed via developer platforms, including Gemini-2.5-Pro-Thinking [39], OpenAI O3 [43], GPT-5 [42], and Claude-Sonnet-4.5 [41]. **b) General-Purpose Open-Source:** Models such as Qwen3-VL [44], GLM-4.5 [46], and Step-3 [45], also accessed via cloud APIs. **c) GUI-Grounding-Oriented Open-Source:** Models including UI-Tars-1.5 [31] (based on Qwen2.5-VL [47]), UGround [10], and OS-Atlas [52] (both based on Qwen2-VL [47]). These are deployed on an A100 GPU via HuggingFace Inference Endpoints². All models are evaluated at their native resolutions using performance-tuned prompts (see Appendix Sec. 7).

Evaluation Metrics. Grounding is measured by IoU and Grounding Center Accuracy (Acc.) [6, 15], where $\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\text{pred}_i \text{ inside GT bbox}_i) \times 100$ where $\mathbf{1}$ is an indicator function and N the number of test samples. This metric represents the percentage of samples with the predicted points falling in the ground truth bounding boxes. For captioning tasks, we report standard multi-choice question answering accuracy [25, 60].

To analyze performance under varying visual complexity, we decompose metrics by region/action type and propose the **Normalized Interference Density (NID)**. NID quantifies the visual clutter surrounding a target e_{target} with bounding box B_{target} (width W , height H). We first define an “analysis region” R_{analysis} , centered on the target, with a scaled width $W_R = W \cdot (1 + 2\alpha)$ and height $H_R = H \cdot (1 + 2\alpha)$, where $\alpha = 1.0$ is a fixed expansion factor. NID is the count of other element centers e_j within this analysis region:

$$\text{NID} = \sum_{e_j \in E \setminus \{e_{\text{target}}\}} \mathbf{1}(\text{center}(e_j) \in R_{\text{analysis}}).$$

We use NID tertiles to objectively classify samples into **Sparse**, **Medium**, and **Dense** groups, enabling a fine-grained analysis of model robustness to visual interference.

4.2. Benchmarking Results

4.2.1. Region-Level Functionality Understanding

To evaluate the models’ capability in parsing macro-scale GUI structures and functional areas, we conduct assessments

²<https://endpoints.huggingface.co/>

Table 3. **Region grounding performance and decomposition by region type and NID score.** Open-source models (GLM-4.5, Qwen3-VL) dominate commercial leaders. The benchmark’s challenge is confirmed by a universal performance drop when grounding by functionality instead of description. We also note a clear model preference for common regions types (e.g., Global Navigation) and a split NID preference (commercial models favor ‘Dense’ context, open-source favor ‘Sparse’). Moreover, the lower IoU indicates that accurately outputting region boxes is significantly harder. (UI-tars and UGround do not possess a box prediction ability).

Model	Func. Gnd. IoU.	Desc. Gnd. Acc.	Func. Gnd. Acc.	Func. Gnd. Acc. by Region Type						Func. Gnd. Acc. by NID		
				Primary Containers	Global Navigation	Content Display	Interaction Controls	Contextual Overlays	Others	Sparse	Medium	Dense
Gemini-2.5-Pro-Thinking [39]	20.5	77.8	77.6	88.5	79.6	74.3	84.0	71.0	62.2	77.5	76.0	79.7
O3 [43]	17.4	49.2	36.7	53.8	34.3	34.3	44.0	61.3	22.2	17.0	33.5	61.7
GPT-5 [42]	12.8	36.7	31.4	38.5	32.5	34.3	28.0	32.3	20.0	22.5	31.1	41.4
Claude-Sonnet-4.5 [41]	12.7	50.7	48.6	53.8	50.4	48.6	52.0	51.6	31.1	50.0	49.1	46.6
Qwen3-VL-32B-Instruct [44]	23.3	92.3	84.4	92.3	84.6	85.7	84.0	90.3	73.3	82.4	85.6	85.0
Qwen3-VL-8B-Instruct [44]	15.4	87.3	70.1	88.5	69.6	68.6	76.0	77.4	55.6	62.7	69.5	78.9
Step-3 [45]	11.0	47.5	36.7	34.6	40.0	28.6	44.0	29.0	24.4	53.5	29.3	27.8
GLM-4.5V [46]	16.0	93.7	84.6	92.3	87.1	77.1	80.0	90.3	68.9	87.3	85.0	81.2
UI-Tars-1.5 [31]	–	78.3	75.8	84.6	76.8	68.6	72.0	90.3	62.2	74.6	77.8	74.4
UGround-V1-7B [10]	–	88.5	64.7	50.0	66.1	60.0	72.0	83.9	51.1	63.4	62.3	69.2
OS-Atlas-Base-7B [52]	12.4	72.4	56.3	61.5	52.9	62.9	68.0	83.9	44.4	47.2	50.9	72.9

Table 4. **Region captioning multi-choice question accuracy.** In contrast to grounding, top-tier commercial VLMs (e.g., GPT-5, O3) dominate functionality captioning tasks, surpassing open-source models. Higher error rates on ‘Hard’ negatives confirm models are ‘tricked’ by context, not failing randomly.

Model	Outcome Pred. Acc.	Acc. by Region Type						Error Rate by Difficulty	
		Primary Containers	Global Navigation	Content Display	Interaction Controls	Contextual Overlays	Others	Easy	Hard
Gemini-2.5-Pro-Thinking [39]	86.2	79.7	87.0	84.5	87.8	90.6	83.5	4.0	9.8
O3 [43]	84.0	90.3	84.9	74.0	88.5	85.9	78.7	5.9	10.1
GPT-5 [42]	88.1	96.6	88.6	80.8	94.0	90.7	81.2	2.8	9.1
Claude-Sonnet-4.5 [41]	84.0	89.5	84.5	71.6	96.0	85.7	79.8	5.4	10.6
Qwen3-VL-32B-Instruct [44]	80.1	82.8	79.9	70.3	90.0	75.0	85.1	8.2	11.7
Qwen3-VL-8B-Instruct [44]	66.3	70.5	61.7	77.1	77.8	80.4	69.6	11.9	21.8
GLM-4.5V [46]	86.8	94.5	87.7	80.5	89.8	92.5	77.3	4.6	8.6

Table 5. **Element grounding accuracy for the three referring expression types provided by AutoGUI-v2.** Functionality-oriented grounding presents a greater challenge, harder than existing description/intent grounding benchmarks.

Model	AutoGUI-v2 (ours)			ScreenSpot-v2 (Appearance)	OSWorld-G (Intent)
	Functionality	Appearance	Intent		
Gemini-2.5-Pro-Thinking [39]	14.9	18.0	17.4	90.0	39.0
O3 [43]	3.5	4.4	5.0	30.5	9.6
GPT-5 [42]	2.0	4.1	4.8	26.5	7.8
Claude-Sonnet-4.5 [41]	6.4	7.3	6.9	16.8	24.7
Qwen3-VL-32B-Instruct [44]	71.1	80.7	81.9	95.7	65.1
Qwen3-VL-8B-Instruct [44]	57.1	69.5	74.1	94.8	58.2
Qwen2-VL-7B-Instruct [47]	12.4	15.0	17.1	4.5	58.2
Hcompany/Holo2-8B [7]	32.0	36.3	37.9	59.2	70.1
GLM-4.5V [46]	67.4	75.4	74.5	91.8	50.0
Step-3 [45]	5.4	5.7	5.4	15.8	10.4
OpenCUA-72B [49]	67.9	77.1	76.8	92.9	59.2
OpenCUA-32B [49]	55.4	67.9	68.6	93.4	59.6
OpenCUA-7B [49]	52.2	72.1	71.7	92.3	55.3
UI-Venus-Ground-72B [11]	63.4	70.4	70.9	95.3	69.8
UI-Venus-Ground-7B [11]	42.1	54.6	58.4	94.1	58.8
GUI-R1-7B [29]	54.1	68.9	70.0	88.1	42.7
InfGUI-G1-7B [24]	56.6	69.4	71.1	93.5	48.0
UI-Tars-1.5 [31]	54.1	68.9	70.0	94.2	56.8
UGround-V1-7B [10]	14.2	18.9	22.3	76.5	42.4
OS-Atlas-Base-7B [52]	20.8	28.7	31.3	85.1	27.7

on region-level grounding and captioning tasks. The results are summarized in Tab. 3 and Tab. 4.

Open-source VLMs specialized in GUIs dominate region grounding. Specialized open-source models demonstrate a significant advantage in localizing functional regions. As shown in Tab. 3, Qwen3-VL-32B-Instruct [44] and GLM-

4.5 [46] achieve remarkable functionality grounding accuracies of 84.4% and 84.6%, respectively. In stark contrast, general-purpose commercial models struggle significantly in this metric; for instance, GPT-5 [42] only attains 31.4%, and even the robust Gemini-2.5-Pro-Thinking [39] (77.6%) trails the leading open-source models. This performance gap suggests that while commercial models possess strong general capabilities, they may lack the specific ability to pinpoint large-scale GUI regions, whereas large-scale fine-tuning with GUI data appears advantageous.

Functionality-oriented grounding presents a greater challenge than description-oriented grounding. Comparing the “Desc. Gnd. Acc.” and “Func. Gnd. Acc.” columns in Tab. 3 reveals a consistent performance drop across almost all models when shifting from visual descriptions to functionality. For example, UGround-V1-7B [10] drops from 88.5% to 64.7%, and OS-Atlas-Base-7B [10] drops from 72.4% to 56.3%. This trend indicates that mapping explicit visual attributes (e.g., “the blue sidebar”) to coordinates is inherently less challenging than deducing regions from abstract functional descriptions, which requires a deeper level of semantic processing beyond mere visual matching.

Standardized regions are easier to localize than Others regions. Breaking down performance by the region types exemplified in Fig. 1, Tab. 3 shows that the models consistently achieve high accuracy on standardized regions such as Primary Containers and Interaction Controls.

Table 6. **Element grounding accuracy (%) decomposed by action type and the proposed NID score.** Accuracy remains low for complex actions (e.g., Double-Click and Drag). Moreover, performance peaks at ‘Medium’ density (NID), suggesting that such density provides sufficient local context to disambiguate function without overwhelming the model.

Model	Func. Gnd. Acc.	Acc. by Action Type							Acc. by Density Indicator NID		
		Left-Click	Hover	Drag	Right-Click	Double-Click	Long-Press	Type	Sparse	Medium	Dense
Gemini-2.5-Pro-Thinking [39]	14.9	22.0	12.5	8.5	13.6	4.9	31.7	32.4	11.5	23.5	17.4
O3 [43]	3.5	4.0	4.0	0.6	1.1	3.7	11.9	8.8	2.3	4.7	4.9
GPT-5 [42]	2.0	2.5	2.0	0.6	0.0	1.2	7.1	5.9	1.1	3.5	2.9
Claude-Sonnet-4.5 [41]	6.4	12.1	3.0	4.6	1.1	0.0	33.3	0.0	4.9	19.8	4.8
Qwen3-VL-32B-Instruct [44]	71.1	71.5	71.8	34.5	40.0	<u>78.3</u>	87.5	78.6	64.9	69.3	79.2
Qwen3-VL-8B-Instruct [44]	57.1	55.9	57.9	36.1	61.9	87.0	81.3	57.1	49.6	57.9	63.7
Qwen2-VL-7B-Instruct [47]	12.2	11.2	8.5	5.7	22.2	40.0	77.8	33.3	6.5	12.1	18.8
Hcompany/Holo2-8B [7]	32.6	32.0	31.6	19.4	40.0	54.5	60.0	42.9	16.5	40.1	41.0
Step-3 [45]	5.4	5.6	4.5	2.8	1.5	4.3	15.9	14.3	3.1	6.0	7.1
GLM-4.5V [46]	24.4	34.0	27.5	15.3	17.0	13.4	35.7	32.4	21.8	36.0	24.8
OpenCUA-72B [49]	<u>67.9</u>	<u>67.8</u>	<u>68.7</u>	47.1	68.4	76.2	<u>86.7</u>	69.2	64.2	64.2	75.4
OpenCUA-32B [49]	55.4	55.6	57.0	38.9	52.4	73.9	31.2	50.0	48.7	51.1	66.4
OpenCUA-7B [49]	52.2	50.2	54.5	38.9	57.1	69.6	81.3	50.0	44.5	49.7	62.5
UI-Venus-Ground-72B [11]	63.4	65.1	60.4	41.7	<u>66.7</u>	73.9	87.5	<u>71.4</u>	65.4	57.9	67.0
UI-Venus-Ground-7B [11]	42.1	42.1	40.8	30.6	28.6	60.9	81.3	50.0	38.0	41.1	47.3
GUI-R1-7B [29]	45.4	46.8	41.0	36.1	38.1	73.9	87.5	35.7	37.6	45.6	53.0
InfGUI-G1-7B [24]	56.6	55.7	56.7	55.6	38.1	78.3	81.3	57.1	51.5	54.6	63.7
UI-Tars-1.5 [31]	54.1	55.8	52.5	<u>44.4</u>	47.6	60.9	68.8	28.6	50.0	54.1	58.3
UGround-V1-7B [10]	14.2	14.4	10.4	16.7	9.5	21.7	68.8	28.6	12.7	12.6	17.5
OS-Atlas-Base-7B [10]	20.8	21.2	15.7	22.9	28.6	30.4	81.3	28.6	13.9	18.6	29.9

Table 7. **Element-level functionality captioning multi-choice question accuracy.** Commercial VLMs lead in this interaction prediction task, revealing a divergence from models fine-tuned for grounding. Additionally, performance correlates with an action’s visual feedback, succeeding on overt actions (Type) but failing on subtle ones (Hover). Critically, higher error rates on ‘Hard’ negatives than ‘Easy’ ones indicate a gap in context-aware functionality reasoning.

Model	Outcome Pred. Acc.	Acc. by Action Type							Error Rate by Difficulty	
		Left-Click	Hover	Drag	Right-Click	Double-Click	Long-Press	Type	Easy	Hard
Gemini-2.5-Pro-Thinking [39]	70.3	<u>66.0</u>	<u>61.5</u>	<u>71.0</u>	80.0	69.6	<u>76.9</u>	89.3	11.7	18.0
O3 [43]	65.4	59.5	54.5	68.5	<u>70.9</u>	62.5	84.6	<u>90.3</u>	16.6	18.0
GPT-5 [42]	66.5	59.0	56.0	<u>71.0</u>	65.5	<u>71.4</u>	84.6	91.4	16.6	16.9
Claude-Sonnet-4.5 [41]	<u>66.9</u>	59.5	63.5	<u>71.0</u>	69.1	57.1	84.6	86.0	15.4	17.8
Qwen3-VL-32B-Instruct [44]	61.3	57.5	50.5	72.2	50.9	67.9	38.5	79.6	15.0	23.7
Qwen3-VL-8B-Instruct [44]	57.1	58.0	48.5	59.9	47.3	64.3	46.2	72.0	15.7	27.2
Step-3 [45]	52.8	43.5	47.0	56.8	65.5	60.7	38.5	65.6	21.4	25.8
GLM-4.5 [46]	64.8	67.0	55.0	69.1	58.2	73.2	30.8	76.3	14.6	20.6

However, regarding the Others category (e.g., color pickers and logos)—which typically comprises irregular layout regions—we observe model performance generally drops by at least 5% compared with the best-performing region types. Interestingly, Claude-Sonnet-4.5 [41] exhibits strong performance on Interaction Controls, suggesting that while it may struggle with broader regions, it retains strong capabilities in recognizing interactive widgets, such as filters, search regions, and paginations.

Density preference varies by model type. The NID analysis in Tab. 3 reveals an interesting divergence. Commercial models (e.g., O3 [43], GPT-5 [42]) and GUI-specialized models (e.g., OS-Atlas-Base-7B [52]) generally exhibit their highest accuracy in Dense environments (e.g., O3 [43] achieves 61.7% in Dense vs. 17.0% in Sparse). In contrast, general-purpose open-source models (e.g., Step-3 [45],

GLM-4.5 [46]) tend to perform best in Sparse scenarios. This phenomenon probably suggests that commercial and domain-specific models might benefit from the richer contextual cues provided by neighboring layout structures in dense settings, whereas general open-source models seem to prefer cleaner visual inputs with less distractor interference. For a more detailed analysis of this trend, please refer to the Appendix Sec. 7.

Hard negatives reveal robustness gaps in functional reasoning. To verify the rigor of our evaluation, we analyze the Error Rate by Difficulty in Tab. 4. Across all models, the error rate on Hard samples (distractors from the same similarity group) is consistently higher than on Easy samples. For instance, GPT-5 [42] has a Hard error rate of 9.1% versus an Easy error rate of 2.8%. This persistent gap indicates that models are not failing randomly; they are more frequent

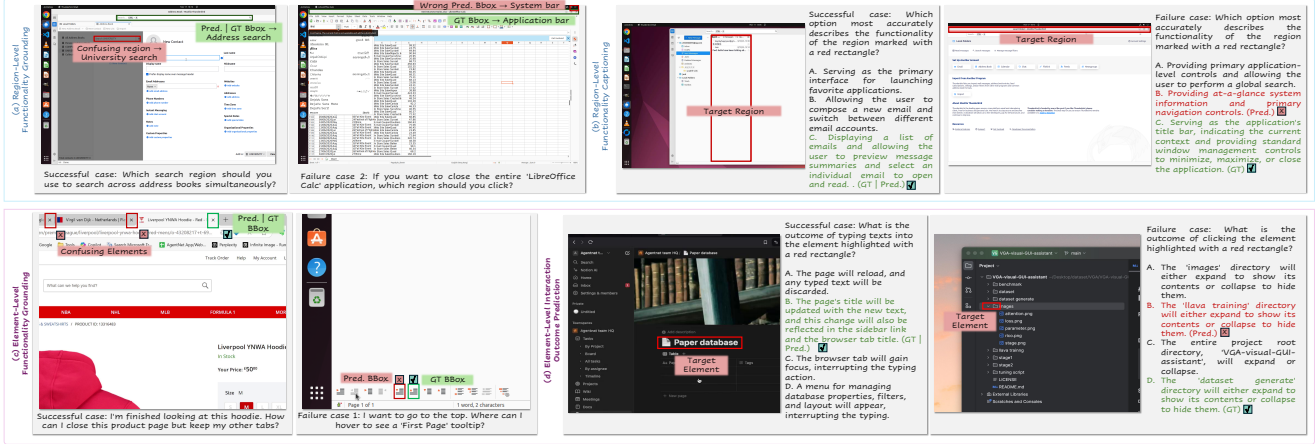


Figure 5. **Functionality understanding cases of Gemini-2.5-Pro-Thinking.** This VLM successfully understand common GUI targets (e.g., search region) but fails on complex or stateful ones (e.g., long software status bar).

to be “tricked” by functionally plausible distractors, highlighting that functional discrimination remains a non-trivial challenge even at the region level.

4.2.2. Element-Level Functionality Understanding

The VLMs are evaluated on our element-level tasks to assess fine-grained functional understanding. The results are presented in Tables 5, 6, and 7.

Opensource VLMs lead in grounding accuracy. Overall grounding accuracy on functionality-based tasks is low (Tab. 5), indicating a significant challenge. Surprisingly, the open-source GLM-4.5 [46] emerges as the top performer, surpassing leading commercial models like Gemini-2.5-Pro-Thinking [39]. We attribute this gap to GLM-4.5’s specialized fine-tuning on GUI agent tasks. Similarly, GUI-oriented VLMs (e.g., OS-Atlas) outperform general commercial models, suggesting that large-scale, domain-specific fine-tuning enhances functional understanding, though it remains insufficient to master our benchmark.

Functionality-based grounding is significantly harder than appearance or intent-based grounding. To quantify the benchmark’s difficulty, we test the VLMs on parallel tasks using appearance or action-intent prompts for the same elements. As shown in Tab. 5, performance on functionality-based tasks is consistently lower across all models. This disparity, combined with higher scores on existing benchmarks (ScreenSpot-v2 [52], OSWorld-G [55]), confirms that understanding functionality is a distinct and more difficult challenge than localizing based on appearance or intent cues.

Models struggle with complex and implicit actions. The action-type breakdown (Tab. 6) reveals a clear pattern: models perform best on actions with explicit outcomes, such as Long-Press and Type. Conversely, accuracy plummets for complex or implicit actions, particularly Double-Click (near-zero for most commercial models) and Right-Click.

This suggests models fail when functionality is not immediately obvious. While GLM-4.5 and Qwen3-VL show the strongest overall grounding, their strengths are non-uniform, highlighting inconsistent mastery of action semantics.

NID analysis reveals an increasing trend. The NID breakdown (Tab. 6) shows an overall increasing trend except GLM-4.5V and UI-Venus-Ground-72. For most of the models, when the number of surrounding elements increases (i.e., NID rises), the grounding accuracy for this group also increases. This suggests that element-level functionality understanding likely relies on a rich context, which means determining the functionality of an element in a functional group is easier than that of an isolated element.

Commercial VLMs excel at captioning, revealing a divergence. In sharp contrast to grounding results, top-tier commercial models (Gemini-2.5-Pro-Thinking) dominate the functional captioning task (Tab. 7). This suggests a clear divergence: GUI-specific fine-tuning improves localization (“where is it?”), but the broad world knowledge of large commercial models is more effective for abstractly articulating purpose (“what does it do?”).

Captioning accuracy correlates with action types. The captioning breakdown by action type (Tab. 7) shows models perform best on actions with overt visual state changes (e.g., Type, Long-Press). Hover is a consistent failure point, as its subtle feedback (e.g., a tooltip) provides a weaker signal for models to learn the associated function.

Error analysis shows models fail on plausible distractors. Our multi-choice setup includes “easy” (irrelevant) and “hard” (visually/semantically similar) negatives. Across all models, the Hard error rate is consistently higher than the Easy rate. This demonstrates that models are not failing randomly; they are actively tricked by plausible-but-incorrect functional descriptions (Fig. 5), highlighting a critical gap in nuanced, context-aware reasoning.

4.2.3. Case Visualization

Qualitative analysis of Gemini-2.5-Pro-Thinking (Fig. 5) reveals its capabilities and limitations. For grounding, the VLM excels at common, unambiguous elements like search bars and ‘close’ buttons. However, it falters on components with complex or abstract functions, such as long status bars or the tiny, specialized icons in productivity software.

For functionality captioning, the model accurately identifies outcomes for navigational elements (e.g., an email preview list) but struggles with dynamic, stateful controls. For instance, it fails to predict the behavior of application control bars or the ‘chevron’ icon for a collapsible folder. These cases, with more visualized in the Appendix, indicate that while VLMs grasp common functions, they currently lack a robust, holistic understanding of complex GUI dynamics.

5. Conclusion

We propose AutoGUI-v2, a comprehensive benchmark for evaluating context-aware GUI functionality understanding at both the region and element levels. We benchmark various leading VLMs and provide insightful analysis. Our results reveal a key “grounding-reasoning divergence” and show that a deep understanding of complex interactions and GUI dynamics remains highly challenging for current models. We hope our work will inspire future efforts to improve GUI agents. For social impacts and limitations of our work, please refer to Sec. 9 and Sec. 10.

References

- [1] Amir Bar, Gaoyue Zhou, Danny Tran, Trevor Darrell, and Yann LeCun. Navigation world models. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 15791–15801, 2025. 2
- [2] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, Lawrence Jang, and Zack Hui. Windows agent arena: Evaluating multi-modal os agents at scale, 2024. 3
- [3] Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Dingyu Zhang, Peng Gao, Shuai Ren, and Hongsheng Li. Amex: Android multi-annotation expo dataset for mobile gui agents, 2024. 4, 1
- [4] Wentong Chen, Junbo Cui, Jinyi Hu, Yujia Qin, Junjie Fang, Yue Zhao, Chongyi Wang, Jun Liu, Guirong Chen, Yupeng Huo, Yuan Yao, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Guicourse: From general vision language models to versatile gui agents, 2024. 3
- [5] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, Bin Li, Ping Luo, Tong Lu, Yu Qiao, and Jifeng Dai. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *CVPR*, pages 24185–24198, 2024. 2
- [6] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *ACL*, pages 9313–9332, 2024. 2, 3, 5
- [7] H Company. Holo2 - open foundation models for navigation and computer use agents, 2025. 6, 7
- [8] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *NIPS*, 36, 2024. 2, 3
- [9] Zvi Galil and Giuseppe F. Italiano. Data structures and algorithms for disjoint set union problems. *ACM Comput. Surv.*, 23(3):319–344, 1991. 5
- [10] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *ICLR*, 2025. 2, 3, 5, 6, 7
- [11] Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, et al. Ui-venus technical report: Building high-performance ui agents with rft, 2025. 6, 7
- [12] Wenyi Hong, Wei Han Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents. In *CVPR*, pages 14281–14290, 2024. 2, 3
- [13] Zhiyuan Huang, Ziming Cheng, Juntong Pan, Zhaohui Hou, and Mingjie Zhan. Spiritsight agent: Advanced gui agent with one look. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 29490–29500, 2025. 3
- [14] Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and ZHAOXIANG ZHANG. Sheetcopilot: Bringing software productivity to the next level through large language models. In *Advances in Neural Information Processing Systems*, pages 4952–4984. Curran Associates, Inc., 2023. 2, 3
- [15] Hongxin Li, Jingfan Chen, Jingran Su, Yuntao Chen, Qing Li, and Zhaoxiang Zhang. Autogui: Scaling gui grounding with automatic functionality annotations from llms. In *ACL*, 2025. 2, 3, 4, 5
- [16] Hongxin Li, Jingran Su, Jingfan Chen, Zheng Ju, Yuntao Chen, Qing Li, and Zhaoxiang Zhang. Uipro: Unleashing superior interaction capability for gui agents, 2025. 2, 1
- [17] Kaixin Li, Meng ziyang, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: GUI grounding for professional high-resolution computer use. In *Workshop on Reasoning and Planning for Large Language Models*, 2025. 2, 3, 4, 1
- [18] Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. In *NIPS*, pages 92130–92154. Curran Associates, Inc., 2024. 1
- [19] Y. Li, Gang Li, Luheng He, Jingjie Zheng, Hong Li, and Zhiwei Guan. Widget captioning: Generating natural language description for mobile user interface elements. In *EMNLP*, 2020. 2
- [20] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *CVPR*, pages 19498–19508, 2025. 2, 3

- [21] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NIPS*, pages 34892–34916. Curran Associates, Inc., 2023. 2
- [22] Junpeng Liu, Yifan Song, Bill Yuchen Lin, Wai Lam, Graham Neubig, Yuanzhi Li, and Xiang Yue. Visualwebbench: How far have multimodal LLMs evolved in web page understanding and grounding? In *First Conference on Language Modeling*, 2024. 2, 3
- [23] Yuhang Liu, Zeyu Liu, Shuanghe Zhu, Pengxiang Li, Congkai Xie, Jiasheng Wang, Xueyu Hu, Xiaotian Han, Jianbo Yuan, Xinyao Wang, et al. Infigui-g1: Advancing gui grounding with adaptive exploration policy optimization. *ArXiv preprint*, abs/2508.05731, 2025. 3
- [24] Yuhang Liu, Zeyu Liu, Shuanghe Zhu, Pengxiang Li, Congkai Xie, Jiasheng Wang, Xueyu Hu, Xiaotian Han, Jianbo Yuan, Xinyao Wang, Shengyu Zhang, Hongxia Yang, and Fei Wu. Infigui-g1: Advancing gui grounding with adaptive exploration policy optimization. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(38):32267–32275, 2026. 6, 7
- [25] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations (ICLR)*, 2024. 5
- [26] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices, 2024. 2, 3
- [27] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based gui agent, 2024. 2, 3, 5, 1
- [28] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanqing Xiong, and Hongsheng Li. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. *ArXiv preprint*, abs/2503.21620, 2025. 3
- [29] Run Luo, Lu Wang, Wanwei He, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025. 6, 7
- [30] Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Juan A. Rodriguez, Montek Kalsi, Nicolas Chapados, M. Tamer Özsu, Aishwarya Agrawal, David Vazquez, Christopher Pal, Perouz Taslakian, Spandana Gella, and Sai Rajeswar. UI-vision: A desktop-centric GUI benchmark for visual perception and interaction. In *Forty-second International Conference on Machine Learning*, 2025. 2, 3
- [31] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. Ui-tars: Pioneering automated gui interaction with native agents, 2025. 2, 3, 5, 6, 7
- [32] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control. In *NIPS*, pages 59708–59728. Curran Associates, Inc., 2023. 2
- [33] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2024. 2, 3
- [34] Jonathan Richens, Tom Everitt, and David Abel. General agents need world models. In *Forty-second International Conference on Machine Learning*, 2025. 2, 3
- [35] Chenrui Shi, Zedong Yu, Zhi Gao, Ruining Feng, Enqi Liu, Yuwei Wu, Yunde Jia, Liuyu Xiang, Zhaofeng He, and Qing Li. Gui knowledge bench: Revealing the knowledge gap behind vlm failures in gui tasks, 2025. 2, 3
- [36] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3, 2025. 5
- [37] Yuchen Sun, Shanhui Zhao, Tao Yu, Hao Wen, Samith Va, Mengwei Xu, Yuanchun Li, and Chongyang Zhang. Gui-Explore: Empowering generalizable gui agents with one exploration. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 19477–19486, 2025. 3
- [38] Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, Jun Xiao, and Yueting Zhuang. Gui-g²: Gaussian reward modeling for gui grounding, 2025. 3
- [39] Google DeepMind Team. Gemini 2.5 pro, 2025. 2, 4, 5, 6, 7, 8
- [40] Meta Llama Team. /llama-3.2-11b-vision, 2024. 2
- [41] OpenAI Team. Introducing claudesonnet 4.5, 2025. 5, 6, 7
- [42] OpenAI Team. Gpt-5 is here, 2025. 5, 6, 7
- [43] OpenAI Team. Introducing openai o3 and o4-mini, 2025. 5, 6, 7
- [44] Qwen Team. Qwen3-vl: Sharper vision, deeper thought, broader action, 2025. 2, 5, 6, 7
- [45] StepFun Team. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding, 2025. 5, 6, 7
- [46] V Team, Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Lihang Pan, Shuaiqi Duan, Weihang Wang, Yan Wang, Yean Cheng, Zehai He, Zhe Su, Zhen Yang, Ziyang Pan, Aohan Zeng, Baoxu Wang, Bin Chen, Boyan Shi, Changyu Pang, Chenhui Zhang, Da Yin, Fan Yang, Guoqing Chen, Jiazheng Xu, Jiale Zhu, Jiali Chen, Jing Chen, Jinhao Chen, Jinghao Lin, Jinjiang Wang, Junjie Chen, Leqi Lei, Letian Gong, Leyi Pan, Mingdao Liu, Mingde Xu, Mingzhi Zhang, Qinkai Zheng, Sheng Yang, Shi Zhong, Shiyu Huang, Shuyuan Zhao, Siyan Xue, Shangqin Tu, Shengbiao Meng, Tianshu Zhang,

- Tianwei Luo, Tianxiang Hao, Tianyu Tong, Wenkai Li, Wei Jia, Xiao Liu, Xiaohan Zhang, Xin Lyu, Xinyue Fan, Xuancheng Huang, Yanling Wang, Yadong Xue, Yanfeng Wang, Yanzi Wang, Yifan An, Yifan Du, Yiming Shi, Yiheng Huang, Yilin Niu, Yuan Wang, Yuanchang Yue, Yuchen Li, Yutao Zhang, Yuting Wang, Yu Wang, Yuxuan Zhang, Zhao Xue, Zhenyu Hou, Zhengxiao Du, Zihan Wang, Peng Zhang, Debing Liu, Bin Xu, Juanzi Li, Minlie Huang, Yuxiao Dong, and Jie Tang. Glm-4.5v and glm-4.1v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning, 2025. [5](#), [6](#), [7](#), [8](#)
- [47] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution, 2024. [2](#), [5](#), [6](#), [7](#)
- [48] Xuehui Wang, Zhenyu Wu, JingJing Xie, Zichen Ding, Bowen Yang, Zehao Li, Zhaoyang Liu, Qingyun Li, Xuan Dong, Zhe Chen, Weiyun Wang, Xiangyu Zhao, Jixuan Chen, Haodong Duan, Tianbao Xie, Chenyu Yang, Shiqian Su, Yue Yu, Yuan Huang, Yiqian Liu, Xiao Zhang, Yanting Zhang, Xiangyu Yue, Weijie Su, Xizhou Zhu, Wei Shen, Jifeng Dai, and Wenhai Wang. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents, 2025. [3](#), [4](#), [1](#)
- [49] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Zheng Boyuan, LI PEIHANG, Fangyu Lei, Ruisheng Cao, Yeqiao Fu, Dongchan Shin, Martin Shin, Hu Jiarui, Yuyan Wang, Jixuan Chen, Yuxiao Ye, Danyang Zhang, Yipu Wang, Heng Wang, Diyi Yang, Victor Zhong, Y.Charles, Zhilin Yang, and Tao Yu. OpenCUA: Open foundations for computer-use agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. [4](#), [6](#), [7](#), [1](#)
- [50] Yuqi Wang, Jiawei He, Lue Fan, Hongxin Li, Yuntao Chen, and Zhaoxiang Zhang. Driving into the future: Multiview visual forecasting and planning with world model for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14749–14759, 2024. [2](#)
- [51] Ziwei Wang, Weizhi Chen, Leyang Yang, Sheng Zhou, Shengchu Zhao, Hanbei Zhan, Jiongchao Jin, Liangcheng Li, Zirui Shao, and Jiajun Bu. Mp-gui: Modality perception with mllms for gui understanding. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 29711–29721, 2025. [2](#)
- [52] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: Foundation action model for generalist GUI agents. In *ICLR*, 2025. [2](#), [3](#), [5](#), [6](#), [7](#), [8](#)
- [53] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. [2](#), [3](#)
- [54] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. Scaling computer-use grounding via user interface decomposition and synthesis, 2025. [2](#), [3](#), [4](#), [1](#)
- [55] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. Scaling computer-use grounding via user interface decomposition and synthesis, 2025. [3](#), [8](#)
- [56] Yifan Xu, Xiao Liu, Xueqiao Sun, Siyi Cheng, Hao Yu, Hanyu Lai, Shudan Zhang, Dan Zhang, Jie Tang, and Yuxiao Dong. AndroidLab: Training and systematic benchmarking of android autonomous agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2144–2166, Vienna, Austria, 2025. Association for Computational Linguistics. [3](#)
- [57] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguis: Unified pure vision agents for autonomous GUI interaction. In *ICML*, 2025. [2](#), [3](#)
- [58] Pei Yang, Hai Ci, and Mike Zheng Shou. macosworld: A multilingual interactive benchmark for gui agents, 2025. [3](#)
- [59] Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-ui: Visual grounding for gui instructions, 2024. [2](#)
- [60] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruochi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhui Chen. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9556–9567, 2024. [5](#)
- [61] Bofei Zhang, Zirui Shang, Zhi Gao, Wang Zhang, Rui Xie, Xiaojian Ma, Tao Yuan, Xinxiao Wu, Song-Chun Zhu, and Qing Li. Tongui: Building generalized gui agents by learning from multimodal web tutorials. *arXiv preprint arXiv:2504.12679*, 2025. [2](#)
- [62] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771*, 2023. [3](#)
- [63] Jiwen Zhang, Jihao Wu, Teng Yihua, Minghui Liao, Nuo Xu, Xiao Xiao, Zhongyu Wei, and Duyu Tang. Android in the zoo: Chain-of-action-thought for GUI agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 12016–12031, Miami, Florida, USA, 2024. Association for Computational Linguistics. [2](#)

- [64] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. 2025. [4](#)